

CampusTrade 前端测试任务清单

给前端测试同学的完整任务包。后端测试已收工（322 条 pytest + AI 准确率 + Locust 性能），现在轮到前端。

总工作量预估：4-6 天（如果熟悉 Jest / RTL）/ 6-8 天（边学边做）

CA2 计划要求的部分：Phase 1 + Phase 2 + Phase 3.A 必做；Phase 3.B-D 是 Cypress 4 个 workflow，CA2 计划 Table I 也明确列出来了。

Phase 1 — 测试基建（半天）

1.1 安装依赖

```
cd /Users/wcy/Desktop/campusTrade-main/frontend
npm install -D vitest @vitest/ui jsdom @testing-library/react \
  @testing-library/jest-dom @testing-library/user-event \
  msw @vitejs/plugin-react
```

为什么用 **Vitest** 不用 **Jest**？项目已经用 Vite，Vitest 是 Vite 原生测试框架，零配置接入；API 跟 Jest 几乎完全一样。

1.2 配置 vite.config.js

在 `vite.config.js` 里加 `test` 字段：

```
/// <reference types="vitest" />
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',           // 模拟浏览器环境
    globals: true,                  // 不用 import describe/it/expect
    setupFiles: ['./src/test/setup.js'],
    css: false,                     // 不解析 CSS（提速）
  },
})
```

```
    },  
  })
```

1.3 建 setup 文件

src/test/setup.js :

```
import '@testing-library/jest-dom'  
import { afterEach } from 'vitest'  
import { cleanup } from '@testing-library/react'  
  
afterEach(() => {  
  cleanup() // 每个测试后清掉 DOM  
})  
  
// localStorage mock (前端到处用 localStorage 存 token)  
const storage = {}  
global.localStorage = {  
  getItem: (k) => storage[k] ?? null,  
  setItem: (k, v) => { storage[k] = String(v) },  
  removeItem: (k) => { delete storage[k] },  
  clear: () => { Object.keys(storage).forEach(k => delete storage[k]) },  
}
```

1.4 在 package.json 加脚本

```
{  
  "scripts": {  
    "test": "vitest",  
    "test:ui": "vitest --ui",  
    "test:run": "vitest run",  
    "test:coverage": "vitest run --coverage"  
  }  
}
```

1.5 验证基建跑通

写一个 hello-world 测试 src/__tests__/sanity.test.jsx :

```
import { describe, it, expect } from 'vitest'  
import { render, screen } from '@testing-library/react'
```

```
describe('Sanity', () => {
  it('renders hello', () => {
    render(<div>Hello</div>)
    expect(screen.getByText('Hello')).toBeInTheDocument()
  })
})
```

跑：`npm test`。看到 **1 passed** 就基建完工。

1.6 项目版本注意事项（写测试前必读）

本项目实际用的版本（来自 `package.json`），写测试时几个细节要留意：

库	版本	测试时的注意点
React	19.2.0	比常见教程的 React 18 新； <code>act()</code> 现在从 <code>react</code> 直接 import，不用 <code>react-dom/test-utils</code> 。RTL 自动处理 <code>act</code> 包裹，多数情况无感。
react-router-dom	7.13.0	v7 推荐 <code>createMemoryRouter + RouterProvider</code> ，但 <code>MemoryRouter</code> 仍兼容（本文档代码示例用 <code>MemoryRouter</code> 即可，无需重构）。
Tailwind CSS	3.4.x	Vitest 已通过 <code>css: false</code> 跳过 CSS 解析；测试中不需要 mock Tailwind class， <code>screen.getByText/Role</code> 照常工作。
Vite	7.3.x	与 Vitest 完全配套， <code>import.meta.env</code> 在测试里可用。
ESLint	9.x（flat config）	如果给测试文件加规则，改 <code>eslint.config.js</code> 不是 <code>.eslintrc</code> 。可选装 <code>eslint-plugin-testing-library</code> 做静态检查。

RR v7 实战提示：本文档所有 `import { MemoryRouter } from 'react-router-dom'` 都仍然能用。如果未来想升级到 v7 推荐写法，参考：

```
// 旧（仍可用）
render(<MemoryRouter><Component /></MemoryRouter>)

// 新（v7 推荐，但本项目可暂不切换）
import { createMemoryRouter, RouterProvider } from 'react-router-dom'
const router = createMemoryRouter([[ path: '/', element: <Component /> ]])
```

```
render(<RouterProvider router={router} />)
```

Phase 2 — 单元 / 组件测试 (2-3 天)

2.1 必测：表单校验 (CA2 Aim 1.1 硬指标)

CA2 plan 第 3 页明确写："Test that the registration form correctly validates email format and password requirements before submission."

按优先级测：

文件	重点测什么	用例数
<code>src/pages/Register.jsx</code>	邮箱格式、密码长度 (≥6 / ≥8 看后端要求)、密码确认匹配、提交按钮启用条件	6-8
<code>src/pages/Login.jsx</code>	邮箱格式、必填校验、错误提示展示	4-5
<code>src/pages/ForgotPassword.jsx</code>	邮箱格式、提交后展示成功提示	3
<code>src/pages/ResetPassword.jsx</code>	密码强度、确认密码匹配	3
<code>src/pages/ChangePassword.jsx</code>	旧密码非空、新密码强度	3
<code>src/pages/PublishProduct.jsx</code>	标题非空、价格 > 0、分类必选、图片必传	5
<code>src/pages/EditProduct.jsx</code>	同上 + 修改后保存按钮启用	4

示例：测注册表单邮箱校验

```
// src/__tests__/Register.test.jsx
import { describe, it, expect } from 'vitest'
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { MemoryRouter } from 'react-router-dom'
```

```

import Register from '../pages/Register'

const renderWithRouter = (ui) =>
  render(<MemoryRouter>{ui}</MemoryRouter>)

describe('Register form validation', () => {
  it('rejects non-university email', async () => {
    renderWithRouter(<Register />)
    const emailInput = screen.getByPlaceholderText(/email/i)
    await userEvent.type(emailInput, 'hacker@gmail.com')
    await userEvent.click(screen.getByRole('button', { name: /sign up|register/i }))
    expect(await screen.findByText(/university email/i)).toBeInTheDocument()
  })

  it('rejects password shorter than 6 chars', async () => {
    renderWithRouter(<Register />)
    await userEvent.type(screen.getByPlaceholderText(/password/i), '123')
    await userEvent.click(screen.getByRole('button', { name: /sign up|register/i }))
    expect(await screen.findByText(/password.*least.*6/i)).toBeInTheDocument()
  })

  it('enables submit button when all fields valid', async () => {
    renderWithRouter(<Register />)
    await userEvent.type(screen.getByPlaceholderText(/email/i), 'a@university.edu')
    await userEvent.type(screen.getByPlaceholderText(/username/i), 'alice')
    await userEvent.type(screen.getByPlaceholderText(/password/i), 'Password123')
    expect(screen.getByRole('button', { name: /sign up|register/i })).toBeEnabled()
  })
})

```

2.2 必测：通用组件渲染（CA2 Aim 4.1）

5 个 `src/components/` 下的组件，每个 2-4 条用例：

组件	测什么
<code>ProductCard.jsx</code>	渲染商品标题/价格/图片；点击触发跳转；is_favorited 状态切换
<code>NotificationBell.jsx</code>	红点显示未读数；点击展开列表；空状态
<code>ProtectedRoute.jsx</code>	未登录跳转 /login；已登录渲染子组件

示例：测 **ProductCard**

```
// src/__tests__/ProductCard.test.jsx
import { describe, it, expect } from 'vitest'
import { render, screen } from '@testing-library/react'
import { MemoryRouter } from 'react-router-dom'
import ProductCard from '../components/ProductCard'

const sampleProduct = {
  id: '123',
  title: 'Used Calculus textbook',
  price: 9.99,
  category: 'Textbooks',
  images: ['/images/abc.jpg'],
  is_favorited: false,
}

describe('ProductCard', () => {
  it('shows title and price', () => {
    render(<MemoryRouter><ProductCard product={sampleProduct} /></MemoryRouter>)
    expect(screen.getByText('Used Calculus textbook')).toBeInTheDocument()
    expect(screen.getByText(/9.99/)).toBeInTheDocument()
  })

  it('shows empty heart when not favorited', () => {
    render(<MemoryRouter><ProductCard product={sampleProduct} /></MemoryRouter>)
    expect(screen.getByLabelText(/add to favorites/i)).toBeInTheDocument()
  })

  it('shows filled heart when favorited', () => {
    render(<MemoryRouter>
      <ProductCard product={{...sampleProduct, is_favorited: true}} />
    </MemoryRouter>)
    expect(screen.getByLabelText(/remove from favorites/i)).toBeInTheDocument()
  })
})
```

2.3 推荐：页面渲染冒烟测试（24 个页面）

每个页面 1 条"能不能渲染不报错"的测试。完整列表 + 优先级：

优先级	页面	文件
● P0	Home	Home.jsx
● P0	Login	Login.jsx
● P0	Register	Register.jsx
● P0	ProductDetail	ProductDetail.jsx
● P0	PublishProduct	PublishProduct.jsx
● P0	Chat	Chat.jsx
● P0	MyOrders	MyOrders.jsx
● P1	MeProfile	MeProfile.jsx
● P1	MyFavorites	MyFavorites.jsx
● P1	Conversations	Conversations.jsx
● P1	OrderDetail	OrderDetail.jsx
● P1	EditProduct	EditProduct.jsx
● P1	MyProducts	MyProducts.jsx
● P1	SellerProfile	SellerProfile.jsx
● P1	AdminUsers	AdminUsers.jsx
● P1	AdminProducts	AdminProducts.jsx
● P1	AdminReports	AdminReports.jsx
● P1	AdminReview	AdminReview.jsx
● P2	ChangePassword	ChangePassword.jsx
● P2	ForgotPassword	ForgotPassword.jsx
● P2	ResetPassword	ResetPassword.jsx

● P2	VerifyEmail	VerifyEmail.jsx
● P2	RecentViewed	RecentViewed.jsx
● P2	MyReviews	MyReviews.jsx

```
// src/__tests__/Home.test.jsx
import { describe, it, expect, vi } from 'vitest'
import { render, screen, waitFor } from '@testing-library/react'
import { MemoryRouter } from 'react-router-dom'
import Home from '../pages/Home'

// mock fetch
global.fetch = vi.fn(() => Promise.resolve({
  ok: true,
  json: () => Promise.resolve([
    { id: '1', title: 'Product A', price: 10, images: [], category: 'Other' }
  ]),
}))

describe('Home', () => {
  it('renders product list from API', async () => {
    render(<MemoryRouter><Home /></MemoryRouter>)
    await waitFor(() => {
      expect(screen.getByText('Product A')).toBeInTheDocument()
    })
  })

  it('renders search bar', () => {
    render(<MemoryRouter><Home /></MemoryRouter>)
    expect(screen.getByPlaceholderText(/search/i)).toBeInTheDocument()
  })
})
```

优先级：

1. ● 必做：Home, Login, Register, ProductDetail, PublishProduct, Chat, MyOrders
2. ● 推荐：MeProfile, MyFavorites, Conversations, OrderDetail, AdminUsers/Products/Reports
3. ● 可选：剩下其他页面

2.4 推荐：响应式断点（CA2 plan 明确要求）

CA2 第 3 页 : "Render ProductCard with mock data → assert title, price, image displayed. Use jest-matchmedia-mock to simulate viewports and assert layout changes appropriately. Mobile (375px), tablet (768px), desktop (1024px)."

```
npm install -D jest-matchmedia-mock
```

```
import MatchMediaMock from 'jest-matchmedia-mock'

let matchMedia

beforeEach(() => { matchMedia = new MatchMediaMock() })
afterEach(() => { matchMedia.clear() })

it('uses mobile layout at 375px', () => {
  matchMedia.useMediaQuery('(max-width: 768px)')
  // ... 渲染并断言
})
```

2.5 必测 : Context 全局状态 (3 个)

这 3 个 Context 被全站组件依赖，挂了影响面最大。每个 4-6 条用例。

2.5.1 context/AuthContext.jsx — 登录状态全局管理

测什么：

场景	期望
启动时 localStorage 有 token → 自动 fetch /me 还原用户	user state 正确
启动时无 token → user=null, loading=false	不触发 /me 请求
setUser(newUser) → 同时更新 state + localStorage	两边同步
logout() → 清空 state + 清 token + 清 user	localStorage 干净
在 Provider 外用 useAuth()	抛错或返 null (防止误用)

示例：

```

// src/__tests__/AuthContext.test.jsx
import { describe, it, expect, beforeEach, vi } from 'vitest'
import { renderHook, act, waitFor } from '@testing-library/react'
import { AuthProvider, useAuth } from '../context/AuthContext'

const wrapper = ({ children }) => <AuthProvider>{children}</AuthProvider>

beforeEach(() => {
  localStorage.clear()
  global.fetch = vi.fn()
})

describe('AuthContext', () => {
  it('returns null user when no token in storage', async () => {
    const { result } = renderHook(() => useAuth(), { wrapper })
    await waitFor(() => expect(result.current.loading).toBe(false))
    expect(result.current.user).toBeNull()
  })

  it('fetches /me when token exists in storage', async () => {
    localStorage.setItem('token', 'fake-jwt')
    global.fetch.mockResolvedValueOnce({
      ok: true,
      json: async () => ({ id: '1', email: 'a@u.edu', username: 'alice' }),
    })
    const { result } = renderHook(() => useAuth(), { wrapper })
    await waitFor(() => expect(result.current.user).not.toBeNull())
    expect(result.current.user.email).toBe('a@u.edu')
  })

  it('logout clears storage and state', async () => {
    localStorage.setItem('token', 'fake')
    const { result } = renderHook(() => useAuth(), { wrapper })
    await waitFor(() => expect(result.current.loading).toBe(false))
    act(() => result.current.logout())
    expect(localStorage.getItem('token')).toBeNull()
    expect(result.current.user).toBeNull()
  })
})

```

2.5.2 context/UnreadContext.jsx — 未读消息计数

测什么：

场景	期望
已登录用户挂载时 → 调 GET /messages/unread-count	unreadCount 来自接口
未登录 → unreadCount = 0，不发请求	节省网络
收到 WebSocket unread_update 消息	unreadCount 实时更新
用户登出 → unreadCount 重置为 0	不残留旧用户的数据
测试时需要 mock useAuth() 和 useWs() 两个依赖的 hook。	

2.5.3 context/WebSocketContext.jsx — WS 连接管理

测什么：

场景	期望
Provider 挂载后 useWs() 返回连接对象	可访问 isConnected / sendMessage
Provider 外调 useWs()	抛错或返 null
内部用了 useWebSocket("/ws")	hook 被调
这个相对简单，主要是 wrapper 转发行为。	

2.6 必测：核心工具函数

2.6.1 src/api.js — 全站 HTTP 出口

这是整个前端最重要的工具文件，测它就是测所有 API 调用的基石。

测什么：

函数	场景	期望
API_BASE	设了 VITE_API_URL 环境变量	用环境变量
API_BASE	dev 环境下没设环境变量	用 /api (走 Vite proxy)

<code>WS_BASE</code>	http 转 ws / https 转 wss	协议正确替换
<code>getStoredToken()</code>	localStorage 有 token	优先返 localStorage
<code>getStoredToken()</code>	只有 sessionStorage 有	fallback 到 sessionStorage
<code>setStoredUser(user)</code>	写入用户信息	localStorage 中可读到
<code>authFetch(url, opts)</code>	已登录	自动加 <code>Authorization: Bearer <token></code> 头
<code>authFetch(url, opts)</code>	未登录	不加 Authorization 头
<code>authFetch(...)</code>	后端返 401	自动清 token + 跳转 /login (如有此设计)

示例：

```
// src/__tests__/api.test.js
import { describe, it, expect, beforeEach, vi } from 'vitest'
import { getStoredToken, setStoredUser, authFetch, API_BASE } from '../api'

beforeEach(() => {
  localStorage.clear()
  sessionStorage.clear()
  global.fetch = vi.fn().mockResolvedValue({
    ok: true,
    json: async () => ({}),
  })
})

describe('api.js', () => {
  it('getStoredToken returns localStorage token first', () => {
    localStorage.setItem('token', 'local-jwt')
    sessionStorage.setItem('token', 'session-jwt')
    expect(getStoredToken()).toBe('local-jwt')
  })

  it('getStoredToken falls back to sessionStorage', () => {
    sessionStorage.setItem('token', 'session-jwt')
    expect(getStoredToken()).toBe('session-jwt')
  })
})
```

```

it('authFetch attaches Authorization header when logged in', async () => {
  localStorage.setItem('token', 'my-jwt')
  await authFetch('/test')
  const callArgs = global.fetch.mock.calls[0][1]
  expect(callArgs.headers.Authorization).toBe('Bearer my-jwt')
})

it('authFetch does not attach Authorization when no token', async () => {
  await authFetch('/test')
  const callArgs = global.fetch.mock.calls[0][1]
  expect(callArgs?.headers?.Authorization).toBeUndefined()
})
})

```

2.6.2 src/utils/authRedirect.js

测：

```

import { describe, it, expect, vi } from 'vitest'
import { redirectToLogin } from '../utils/authRedirect'

describe('redirectToLogin', () => {
  beforeEach(() => {
    global.confirm = vi.fn(() => true) // 用户点 OK
  })

  it('navigates to /login with from state', () => {
    const navigate = vi.fn()
    const location = { pathname: '/products/123', search: '?ref=home' }
    redirectToLogin(navigate, location)
    expect(navigate).toHaveBeenCalledWith('/login', {
      state: { from: { pathname: '/products/123?ref=home' } },
    })
  })

  it('does not navigate when user cancels confirm', () => {
    global.confirm = vi.fn(() => false)
    const navigate = vi.fn()
    redirectToLogin(navigate, { pathname: '/' })
    expect(navigate).not.toHaveBeenCalled()
  })
})

```

2.7 推荐：App.jsx 路由 smoke test

只测"路由表配对了", 不深入业务：

```
// src/__tests__/App.test.jsx
import { describe, it, expect } from 'vitest'
import { render, screen } from '@testing-library/react'
import { MemoryRouter } from 'react-router-dom'
import App from '../App'

// mock 所有 context 让 App 能跑
vi.mock('../context/AuthContext', () => ({
  AuthProvider: ({ children }) => children,
  useAuth: () => ({ user: null, loading: false }),
}))

describe('App routing', () => {
  it('renders Login at /login', () => {
    render(<MemoryRouter initialEntries={['/login']}><App /></MemoryRouter>)
    expect(screen.getByRole('heading', { name: /login/i })).toBeInTheDocument()
  })

  it('renders Home at /', () => {
    render(<MemoryRouter initialEntries={['/']}><App /></MemoryRouter>)
    expect(screen.getByPlaceholderText(/search/i)).toBeInTheDocument()
  })

  // ...再加 3-5 个关键路由
})
```

2.8 自定义 hook 测试

src/hooks/useWebSocket.js：

```
import { renderHook, act } from '@testing-library/react'
import { describe, it, expect, vi } from 'vitest'
import useWebSocket from '../hooks/useWebSocket'

// mock 全局 WebSocket
class MockWebSocket {
  constructor(url) { this.url = url; this.listeners = {} }
  addEventListener(type, fn) { this.listeners[type] = fn }
```

```

    send(data) { this.lastSent = data }
    close() { this.closed = true }
  }
  global.WebSocket = MockWebSocket

  describe('useWebSocket', () => {
    it('connects with token', () => {
      const { result } = renderHook(() => useWebSocket('fake-token'))
      expect(result.current.socket.url).toContain('fake-token')
    })
  })
})

```

Phase 3 — Cypress 端到端测试 (2-3 天, CA2 plan Table

I)

△ 此 **Phase** 由项目负责人（后端同学）负责，前端测试同学不用做这部分。但你的 Phase 1-2 完成后需要保证以下两点，让 Cypress 能跑：

1. `npm run dev` 在 5173 端口能正常启动（无 console error）
2. 关键页面（Login / Register / Home / PublishProduct / Chat）UI 元素有稳定的可选择器（`data-testid` 优先，或语义化 role/label）

下面 4 个 workflow 是 CA2 Plan Table I 的硬指标，也是 **CA2 Integration Testing** 评分依据。

3.A 安装与配置

```

npm install -D cypress
npx cypress open # 第一次运行会自动建 cypress/ 目录

```

把以下命令加到 package.json：

```

{
  "scripts": {
    "cypress:open": "cypress open",
    "cypress:run": "cypress run",
    "e2e": "cypress run --spec 'cypress/e2e/**/*.cy.js'"
  }
}

```

B.B 4 个 workflow (每个一个 .cy.js 文件)

△ 运行 Cypress 前必须 :

1. 后端跑起来 (`uvicorn main:app --port 8000`)
2. 前端跑起来 (`npm run dev` , 端口 5173)
3. MongoDB 跑起来

Workflow 1: Registration (CA2 Table I 第 1 行)

`cypress/e2e/registration.cy.js` :

```
describe('Registration workflow', () => {
  const testEmail = `test${Date.now()}@university.edu`

  it('user can register, verify email, and reach dashboard', () => {
    // 1. 访问注册页
    cy.visit('/register')

    // 2. 填表单
    cy.get('input[type=email]').type(testEmail)
    cy.get('input[name=username]').type(`user${Date.now()}`)
    cy.get('input[type=password]').type('Password123!')
    cy.contains('button', /register|sign up/i).click()

    // 3. 注册后应跳转到验证邮箱页或登录页
    cy.url().should('match', /verify-email|login/)

    // 4. 后端会真发邮件——CI 测试可以查 DB 拿验证码 :
    // 这里也可以 mock 后端 API 返回固定 token
  })

  it('rejects non-university email with 400', () => {
    cy.visit('/register')
    cy.get('input[type=email]').type('hacker@gmail.com')
    cy.get('input[name=username]').type('hacker')
    cy.get('input[type=password]').type('Password123!')
    cy.contains('button', /register|sign up/i).click()
    cy.contains(/university email/i).should('be.visible')
  })
})
```

验收标准 (CA2 Table I) :

- ✓ 用户被创建, is_verified=false
- ✓ JWT 含合法 user_id
- ✓ 非校园邮箱返回 HTTP 400

Workflow 2: AI Listing (CA2 Table I 第 2 行)

cypress/e2e/ai_listing.cy.js :

```
describe('AI Listing workflow', () => {
  beforeEach(() => {
    // 用已有的测试账号登录
    cy.request('POST', 'http://localhost:8000/auth/login', {
      email: 'perftest@university.edu',
      password: 'PerfTest123!',
    }).then((res) => {
      window.localStorage.setItem('token', res.body.access_token)
    })
  })

  it('user uploads photo → AI generates → product saved', () => {
    cy.visit('/publish')
    cy.get('input[type=file]').selectFile('cypress/fixtures/test_product.jpg')

    // 等 AI 响应 (CA2 要求 < 8s)
    cy.contains(/title|description/i, { timeout: 10000 }).should('be.visible')

    // AI 自动填的字段应可见且可编辑
    cy.get('input[name=title]').should('not.have.value', '')

    cy.contains('button', /publish/i).click()
    cy.url().should('include', '/products/')
  })
})
```

验收标准 :

- ✓ 响应 < 8s
- ✓ AI 字段被表单接收
- ✓ 商品保存成功
- ✓ 未登录用户访问 → HTTP 403

Workflow 3: Message (CA2 Table I 第 3 行)

cypress/e2e/message.cy.js :

```
describe('Message workflow', () => {
  it('verified user can send message; recipient sees it', () => {
    // 登录用户 A
    cy.login('perfctest@university.edu', 'PerfTest123!')
    cy.visit('/products/<some_product_id_owned_by_B>')
    cy.contains('button', /chat|message/i).click()
    cy.get('textarea').type('Hi, is this still available?')
    cy.contains('button', /send/i).click()
    cy.contains('Hi, is this still available?').should('be.visible')
  })

  it('unverified user is blocked from sending', () => {
    cy.login('unverified@university.edu', 'pwd')
    cy.visit('/chat/...')
    cy.contains('button', /send/i).click()
    cy.contains(/verify.*email/i).should('be.visible')
  })
})
```

验收标准：

- ✓ verified 用户：HTTP 200，消息存库
- ✓ unverified 用户：HTTP 403

Workflow 4: Search (CA2 Table I 第 4 行)

cypress/e2e/search.cy.js :

```
describe('Search workflow', () => {
  it('search filters products correctly', () => {
    cy.visit('/')
    cy.get('input[placeholder*="search" i]').type('textbook')
    cy.get('input[placeholder*="search" i]').type('{enter}')
    cy.get('[data-testid=product-card]').each(($el) => {
      cy.wrap($el).invoke('text').should('match', /textbook/i)
    })
  })

  it('combines category and price filters', () => {
    cy.visit('/?category=Electronics&min_price=10&max_price=100')
    cy.get('[data-testid=product-card]').should('have.length.greaterThan', 0)
  })
})
```

```
    })  
  })
```

验收标准：

- ✓ 筛选结果正确
- ✓ 多个 filter 可组合

3.C 自定义命令 (DRY)

`cypress/support/commands.js`：

```
Cypress.Commands.add('login', (email, password) => {  
  cy.request('POST', 'http://localhost:8000/auth/login', { email, password })  
    .then((res) => {  
      window.localStorage.setItem('token', res.body.access_token)  
      window.localStorage.setItem('user', JSON.stringify(res.body.user))  
    })  
})
```

Phase 4 — 输出与报告 (半天)

4.1 跑覆盖率

```
npm run test:coverage  
# 目标：>70% statements/lines (CA2 README 写的目标)
```

4.2 跑 Cypress 全套

```
npm run cypress:run  
# 所有 .cy.js 应通过
```

4.3 准备汇报材料

整理交给项目负责人 (即用户)：

文件	内容
----	----

frontend/coverage/index.html	Vitest 覆盖率报告（截图放进 OA2 报告）
cypress/screenshots/	Cypress 失败截图（如有）
cypress/videos/	Cypress 跑测视频（如开启）
frontend/TESTING_RESULTS.md	写一份汇报：通过率 / 覆盖率 / Cypress 通过情况

完整工作量预估

Phase	内容	时间
1	测试基建	半天
2.1	表单校验测试（7 个表单）	1 天
2.2	通用组件测试（5 个）	半天
2.3	页面冒烟测试（24 个，P0+P1 必做，P2 选做）	1.5 天
2.4	响应式测试	半天
2.5	Context 测试（3 个： Auth / Unread / WebSocket ）	1 天
2.6	核心工具： api.js + authRedirect.js	半天
2.7	App.jsx 路由 smoke test	2 小时
2.8	useWebSocket hook	半天
3	Cypress 4 个 workflow	~~2 天~~ 由项目负责人完成
4	整理报告（Vitest 部分）	半天
合计（前端测试同学）		6-7 天

跟后端的协调点

你需要的	我（后端）已经做好的
已知账号给 Cypress 登录	✓ <code>perfTest@university.edu</code> / <code>PerfTest123!</code> （本地 Mongo <code>campustrade_perf</code> 库）
API 接口契约	✓ FastAPI 自动生成 <code>http://localhost:8000/docs</code>
错误码约定 （400/401/403/404 含义）	✓ 所有路由已统一
后端实测响应时间参考	✓ <code>p95 < 50ms</code> （你 Cypress 等待时间可以放心设短）

后端 322 条 `pytest` + AI 准确率 + Locust 性能测试都通过了，接口行为已经稳定——你写 Cypress 时不用担心后端会乱返回。

有问题问

任何阻塞 ping 项目负责人。后端这块愿意配合：

- 加测试 endpoint
- 调整接口返回格式
- 帮排查 CI 跑不过的问题

加油！🐟